TASK 1

Titanic Survival Prediction

Objective

Build a machine-learning model that predicts whether a Titanic passenger survived.

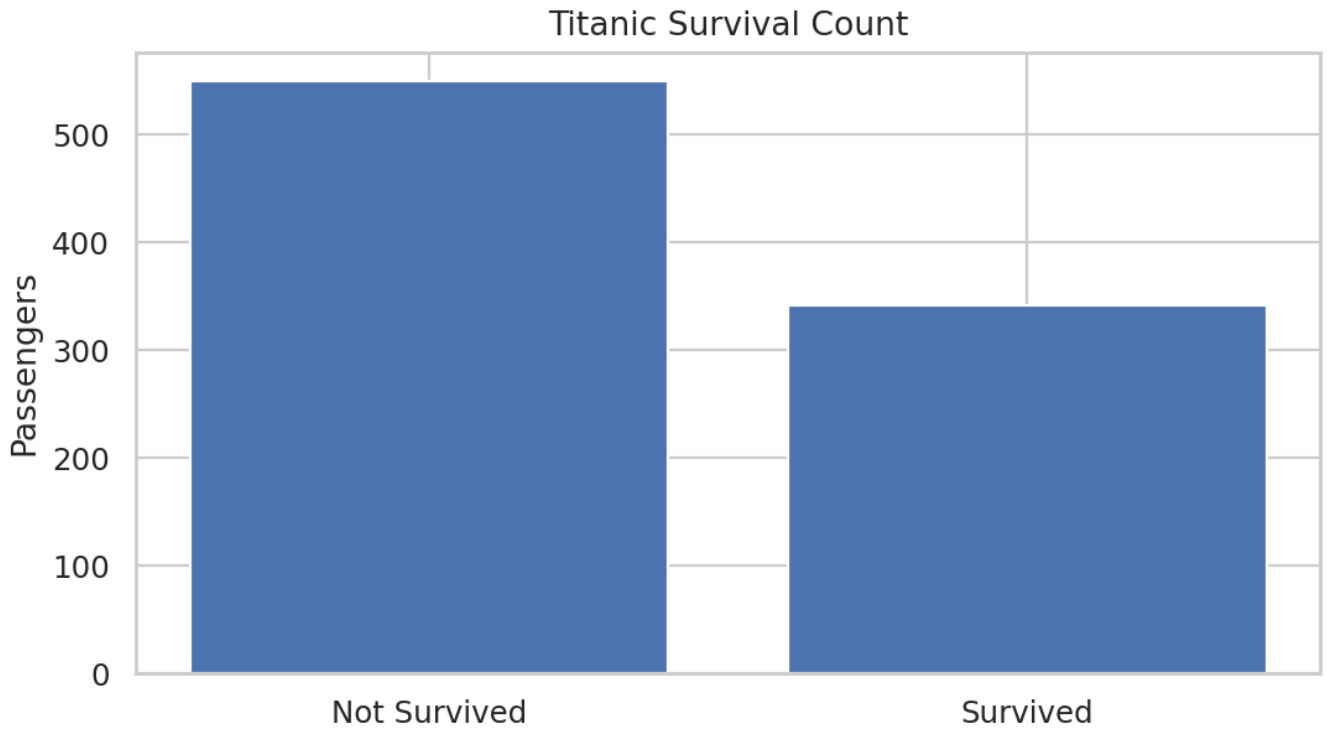
Dataset

The uploaded CodSoft brief defines Task 1 as Titanic Survival Prediction and lists passenger attributes such as age, gender, class, fare and survival.

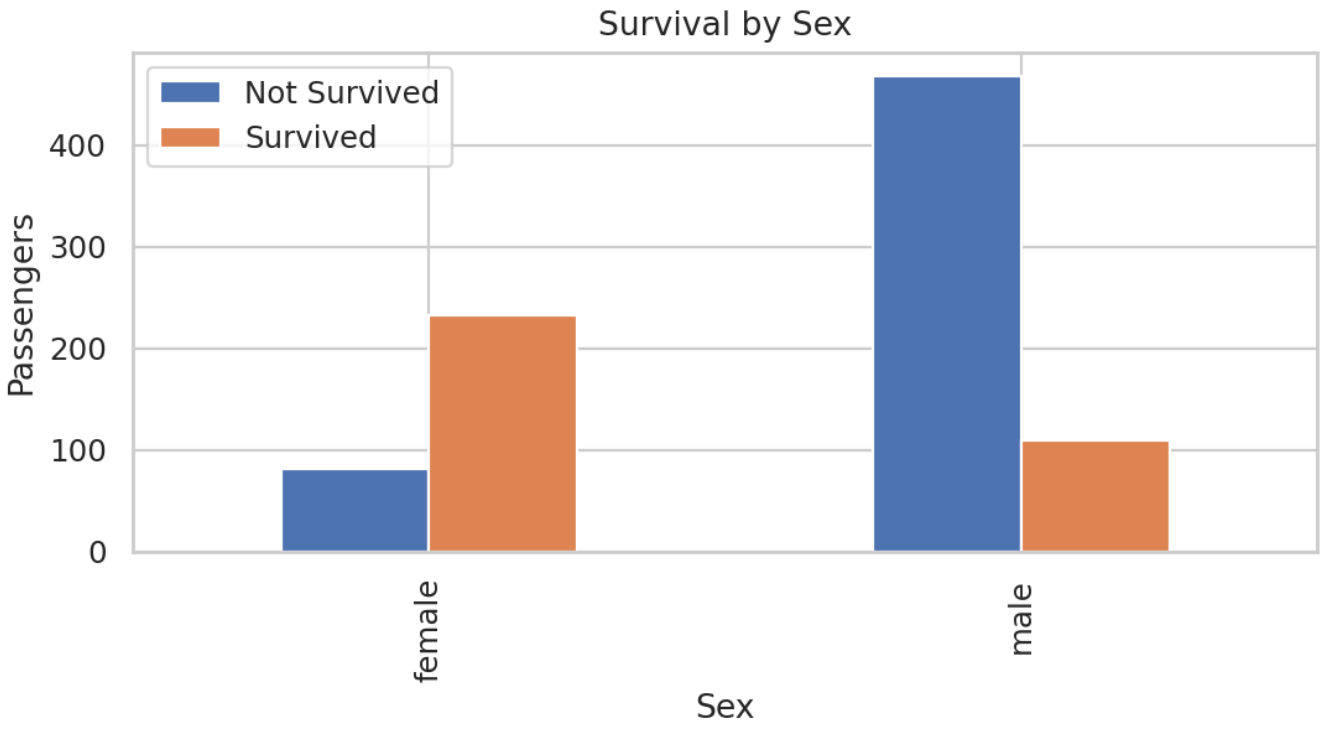
Tools / Libraries

Library	Purpose
Python	Programming
Pandas / NumPy	Data processing
Matplotlib / Seaborn	Data visualization
Scikit-learn	Machine learning

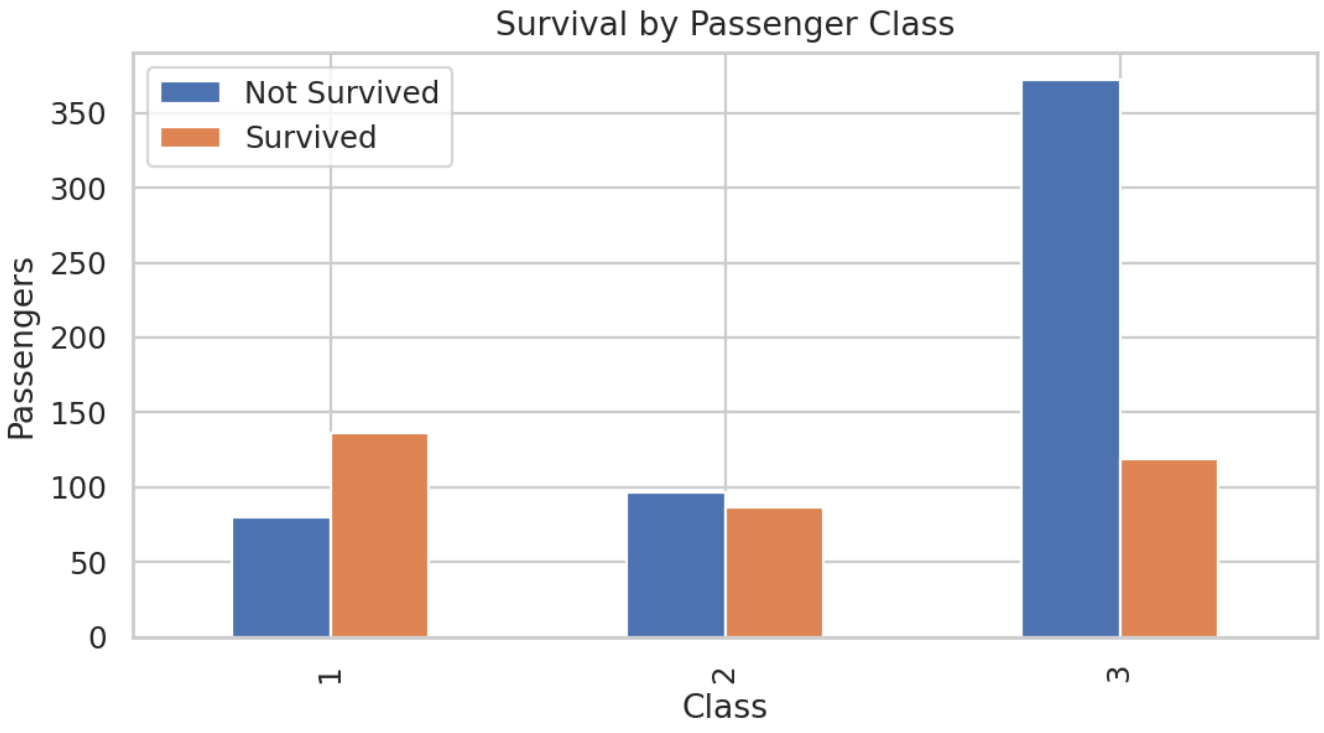
Visualization 1 – Survival Count



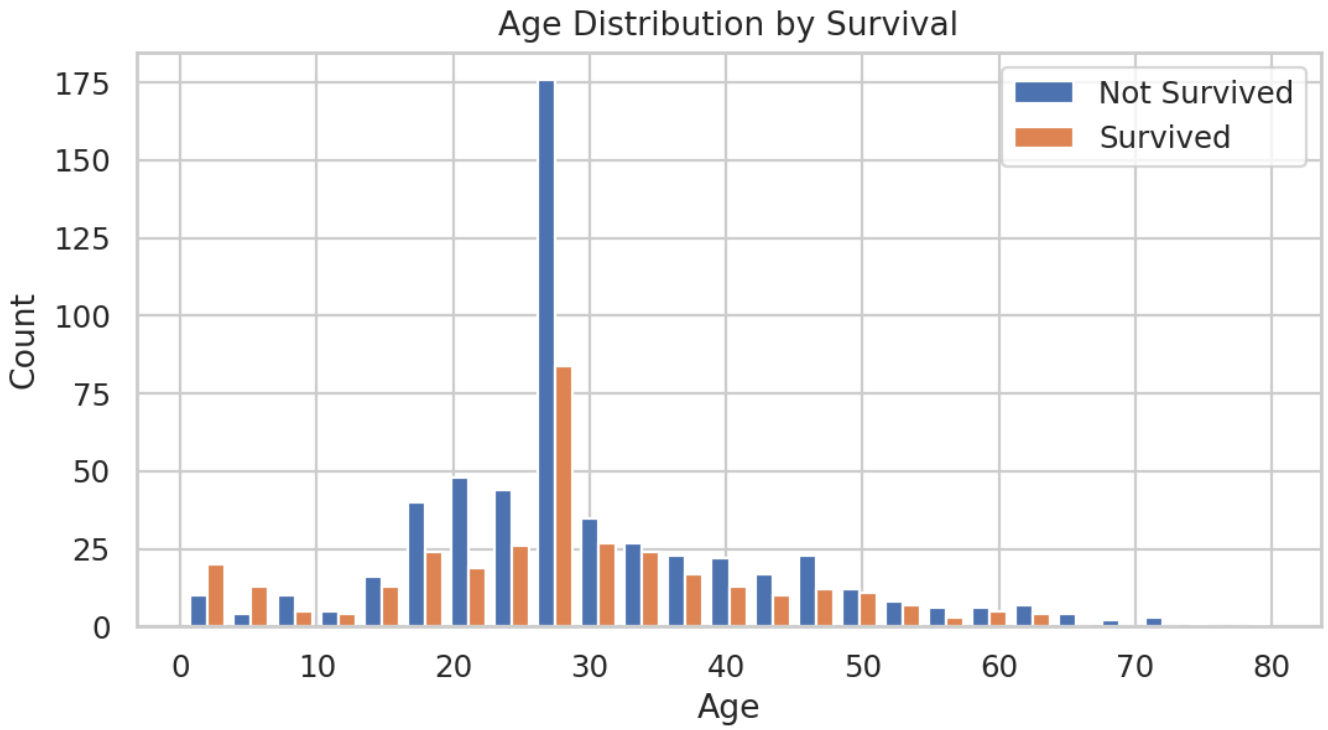
Visualization 2 – Survival by Sex



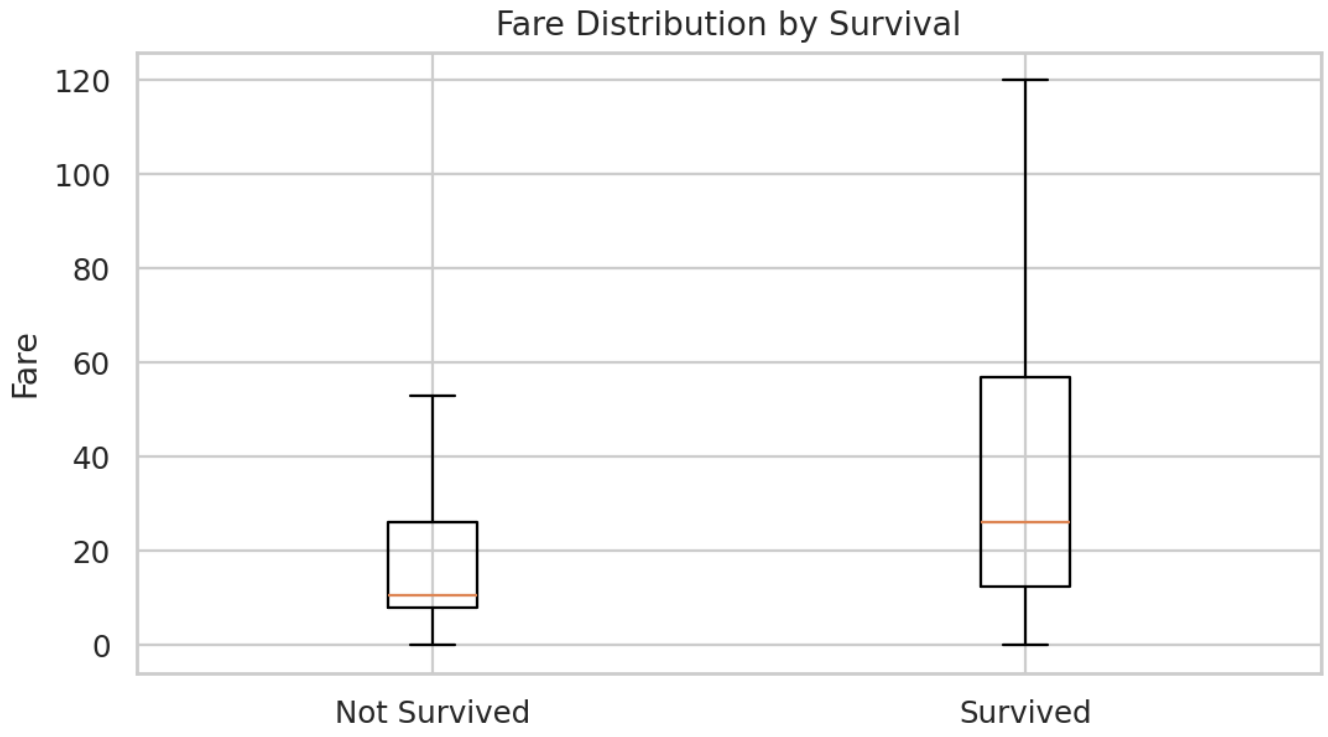
Visualization 3 – Survival by Passenger Class



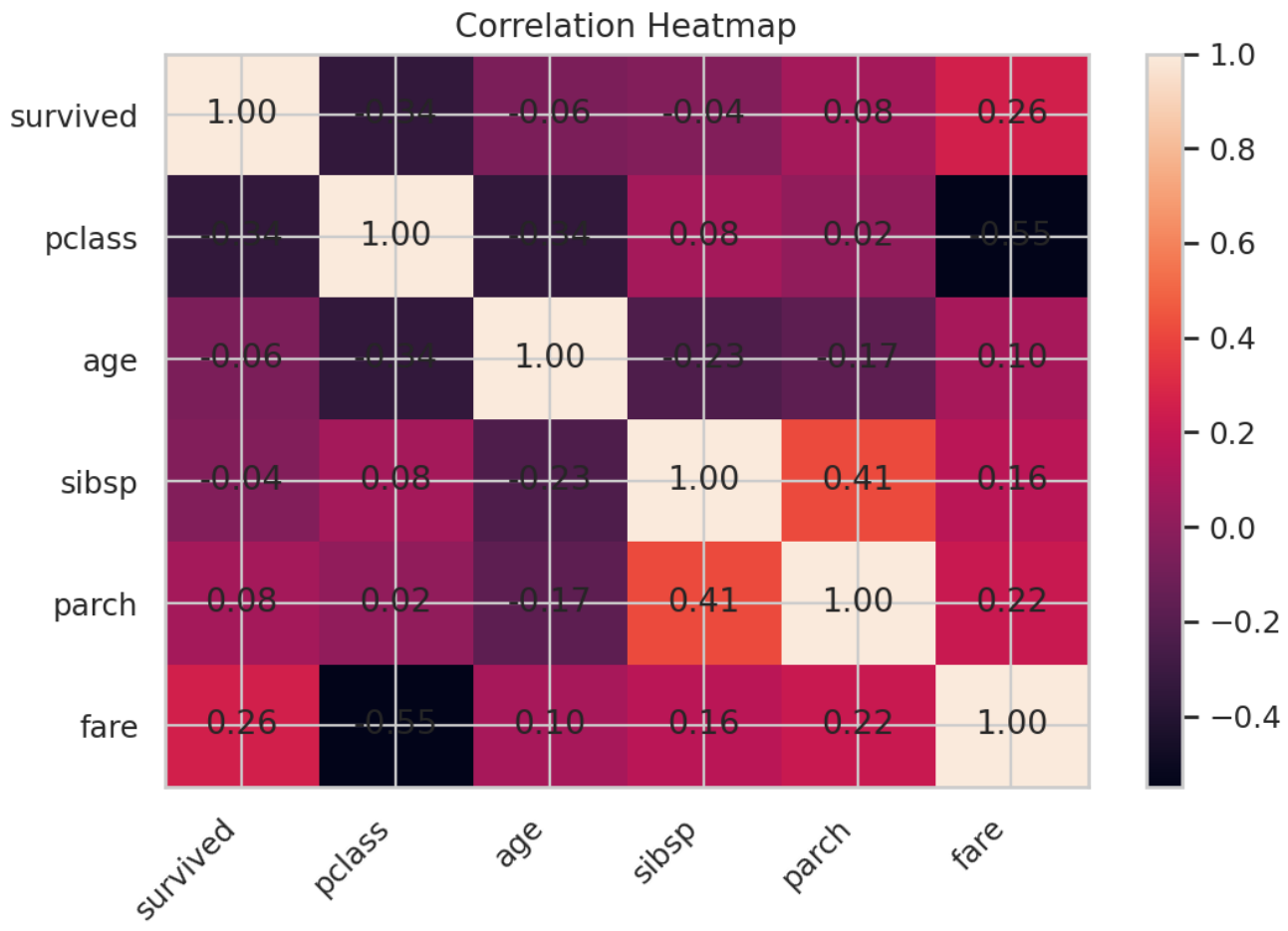
Visualization 4 – Age Distribution



Visualization 5 – Fare Distribution



Visualization 6 – Correlation Heatmap



Model Result

Logistic Regression accuracy: 80.45%. Confusion matrix: [[98, 12], [23, 46]].

Complete Python Code

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv("Titanic-Dataset.csv")
df["Age"] = df["Age"].fillna(df["Age"].median())
df["Embarked"] = df["Embarked"].fillna(df["Embarked"].mode()[0])

# Data Visualizations
sns.countplot(data=df, x="Survived")
plt.title("Titanic Survival Count")
plt.show()

sns.countplot(data=df, x="Sex", hue="Survived")
plt.title("Survival by Sex")
plt.show()

sns.countplot(data=df, x="Pclass", hue="Survived")
plt.title("Survival by Passenger Class")
plt.show()

sns.histplot(data=df, x="Age", hue="Survived", kde=True)
plt.title("Age Distribution by Survival")
plt.show()

sns.boxplot(data=df, x="Survived", y="Fare")
plt.title("Fare Distribution by Survival")
plt.show()

sns.heatmap(df[["Survived","Pclass","Age","SibSp","Parch","Fare"]].corr(),
            annot=True, fmt=".2f")
plt.title("Correlation Heatmap")
plt.show()

# Machine Learning
X=df[["Pclass","Sex","Age","SibSp","Parch","Fare","Embarked"]]
y=df["Survived"]
```

```
X=pd.get_dummies(X,columns=["Sex","Embarked"],drop_first=True)

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score,confusion_matrix,classification_report

X_train,X_test,y_train,y_test=train_test_split(
    X,y,test_size=.20,random_state=42,stratify=y)
model=LogisticRegression(max_iter=1000)
model.fit(X_train,y_train)
y_pred=model.predict(X_test)

print("Accuracy:",accuracy_score(y_test,y_pred))
print(confusion_matrix(y_test,y_pred))
print(classification_report(y_test,y_pred))
```